SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

LexAssist — Legal Intelligence Platform for Indian Law

Version 1.0.0

Status: Production Ready

Date: August 18, 2026

Target Audience: Engineering Team, Legal Operations, Solution Architects, QA Lead, DevOps Engineers

1. Executive Overview & Lifecycle Management

1.1 Document Overview

This document defines the Software Development Life Cycle (SDLC) for LexAssist, a multitasking Legal Intelligence Platform tailored for Indian Law. It covers system requirements, architecture, module design, hallucination auditing, cost-aware model routing, implementation standards, testing, deployment, maintenance, security, and compliance.

1.2 Document Control & Revision History

Version	Date	Author / Role	Summary of Changes
0.1.0	2026-06-10	Engineering Architecture Team	Initial Feasibility Analysis & Technical Specification
0.8.0	2026-07-15	Core Backend Engineers	Implementation of Backbone (VectorStore, Auditor, ModelRouter)
0.9.5	2026-08-01	Full Stack Engineering	Integrated 5 Legal Modules & Vite/React Frontend Interface
1.0.0	2026-08-18	QA & Lead Architect	Production Validation, 32/32 Test Suites Passing, SDLC Finalization

2. Phase 1 — System Vision & Business Feasibility

2.1 Domain Context & Problem Statement

The Indian legal system has high operational complexity, large case backlogs, and a broad regulatory corpus. Legal professionals may need to work across contracts, judicial precedents, case-status information, routine legal drafting, and statutory provisions.

• Contractual Risk Overhead — manual identification of one-sided indemnities, auto-renewal traps, and liability caps.

• Precedent Retrieval Complexity — time-consuming search for judicial rulings and ratio decidendi across disjointed legal search engines.

• Litigation Tracking Latency — difficulty tracking eCourts hearing schedules and case stages across multiple benches.

• Drafting Repetition — inconsistent drafting of routine NDAs, legal notices, and affidavits.

• AI Hallucination Risk — risk of invented case citations, incorrect legal sections, or fabricated judicial precedents.

• API Cost Inflation — uncontrolled LLM consumption when simple queries are routed to expensive flagship models.

2.2 Core Product Vision

LexAssist addresses these challenges through a unified smart intent router, five specialized legal modules, and a shared enterprise backbone.

• Zero-Hallucination Assurance — generated outputs undergo automated self-consistency grounding checks against reference context.

• Deterministic Cost Optimization — query complexity is scored from 0–100 and used to route requests between Lightweight and Flagship model tiers.

• Multi-Module Dispatch — queries are routed to Contract Analysis, Case Law Search, Live Case Status, Document Drafting, or Statutory Explainer.

3. Phase 2 — Requirement Analysis & Technical Specifications

3.1 Functional Requirements

Req ID	Module	Requirement
FR-1.0	Intent Router	Classify natural-language queries into one of five legal modules with confidence scoring and entity extraction.
FR-2.0	Contract Analysis	Ingest contracts, index clauses, flag risks, and answer grounded queries with clause citations.
FR-3.0	Case Law Search	Retrieve precedents, extract ratio decidendi, and summarize judgments with paragraph citations.
FR-4.0	Case Status Tracker	Look up eCourts litigation records by CNR or party name without fabricated facts.
FR-5.0	Document Drafting	Provide validated NDA, Affidavit, and Notice templates and render versioned drafts.

Req ID	Module	Requirement
FR-6.0	Legal Explainer	Explain statutory provisions in plain English with section references.
FR-7.0	Hallucination Auditor	Verify sentence-level overlap and citations against reference context and calculate grounding.
FR-8.0	Cost & Token Telemetry	Calculate token usage, estimate USD/INR costs, and log query telemetry.

3.2 Non-Functional Requirements

• NFR-1 — Reliability & Grounding: average grounding score ≥ 0.85; ungrounded claims trigger Low confidence.

• NFR-2 — Performance & Latency: routing target <100 ms and end-to-end test-session target <1.0 s.

• NFR-3 — Cost Budgeting: complexity score <60 uses the Lightweight tier.

• NFR-4 — Security & Confidentiality: uploaded contracts are processed using ephemeral collections or memory buffers.

• NFR-5 — Resilience: fallback capability when ChromaDB, PyTorch, or third-party LLM APIs are unavailable.

• NFR-6 — Usability: modern dark-themed React UI with telemetry, risk cards, and template wizards.

4. Phase 3 — System Architecture & Technical Design

4.1 System Architecture

React/Vite frontend → FastAPI REST backend → Intent Router → five legal modules → shared backbone services. The backbone contains the vector store, hallucination auditor, cost/model router, and BPE clause tokenizer.

High-level flow: User → Frontend → FastAPI → Intent Router → Legal Module → Backbone → LLM/Vector Store/Fallback → Audited Response.

4.2 Module Deep Dive

4.2.1 Intent Router

Uses regex/keyword scoring and entity detection. It detects 16-character CNR values and statutory section patterns and calculates confidence using the defined scoring formula.

4.2.2 Hallucination Auditor

Extracts citations such as [Clause X], [Section Y], and [Para Z], compares sentence tokens with reference chunks, and calculates a grounding score. The specification uses an overlap threshold of ≥0.35 per sentence.

4.2.3 Cost & Model Router

Scores query complexity from 0–100 using task weights, token count, and legal keyword density. Scores below 60 use Lightweight; scores ≥60 use Flagship. Anthropic/OpenAI integration and structured fallback are supported.

4.2.4 Vector Store Manager

Uses ChromaDB for semantic indexing and falls back to TF-IDF n-gram vectors with cosine similarity when ChromaDB or PyTorch is unavailable.

5. Phase 4 — Development Methodology & Implementation

5.1 Technology Stack

Layer	Technology
Backend	FastAPI 0.110+, Uvicorn 0.28+, Pydantic v2
Language & Runtime	Python 3.10+
Vector Database	ChromaDB 0.4+ with TF-IDF memory fallback
Tokenization	Custom BPE Clause Tokenizer with tiktoken-compatible interface
Frontend	React 18, Vite 4, Lucide-React Icons, Vanilla CSS

Layer	Technology
Testing	Pytest 9.1+, HTTPX TestClient

5.2 Project Structure

LexAssist/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ backbone/
■ ■ ■■■ modules/
■ ■ ■■■ router/
■ ■■■ data/
■ ■■■ tests/
■ ■■■ main.py
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ src/
■ ■■■ package.json
■ ■■■ vite.config.js
■■■ SDLC_DOCUMENT.md

6. Phase 5 — Testing, QA & Verification

6.1 Test Suite Organization & Coverage

Test Module	Coverage	Count	Result
test_api.py	API endpoints	9	PASSED
test_backbone.py	Router, vector store, auditor, cost proxy	9	PASSED
test_modules.py	Five legal modules	14	PASSED
Total	Full System Pipeline	32	100% PASS

6.2 Empirical Test Execution

The supplied execution record reports 32 tests collected and all 32 passing. Result: 32 passed, 1 warning in 0.94s.

7. Phase 6 — Deployment & Operations Strategy

7.1 Backend Setup

cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000

7.2 Frontend Setup

cd frontend
npm install
npm run dev
npm run build

7.3 Environment Variables

Variable	Required	Default	Purpose
ANTHROPIC_API_KEY	Optional	None	Anthropic Claude API access
OPENAI_API_KEY	Optional	None	OpenAI GPT model access
CHROMA_PERSIST_DIR	Optional	./data/chroma_db	Chroma vector storage path
PORT	Optional	8000	FastAPI/Uvicorn service port

8. Phase 7 — Maintenance, Security & Compliance

8.1 Statutory Compliance

• BNS / BNSS Transition — explicit cross-references map historic IPC/CrPC provisions to BNS/BNSS 2023.

• Disclaimer Requirement — module outputs state that LexAssist provides legal information and intelligence, not formal legal advice.

8.2 Security & Privacy Controls

• Ephemeral Ingestion — contract analysis uses short-lived vector identifiers such as contract_<uuid> to isolate party data.

• CORS Policies — main.py supports domain restriction for production deployment.

9. Requirement Traceability Matrix

Requirement	Design Component	Code File	Test Verification	Status
FR-1.0	IntentRouter	intent_router.py	test_backbone.py::test_intent_router	Verified
FR-2.0	ContractAnalysisModule	contract_analysis.py	test_modules.py::test_contract_analysis	Verified
FR-3.0	CaseLawSearchModule	case_law_search.py	test_modules.py::test_case_law_search	Verified
FR-4.0	CaseStatusModule	case_status.py	test_modules.py::test_case_status	Verified
FR-5.0	DocumentDraftingModule	document_drafting.py	test_modules.py::test_document_drafting	Verified
FR-6.0	LegalExplainerModule	legal_explainer.py	test_modules.py::test_legal_explainer	Verified
FR-7.0	SelfConsistencyAuditor	auditor.py	test_backbone.py::test_auditor	Verified
FR-8.0	ModelRouter	cost_proxy.py	test_backbone.py::test_cost_proxy	Verified

10. Sign-Off & Approval

Role	Name / Title	Signature / Status	Date
Lead Systems Architect	Antigravity AI Engineering	Approved	August 18, 2026
QA Lead	Automated Verification Engine	32/32 Passing	August 18, 2026
Legal Operations Lead	LexAssist Product Owner	Approved	August 18, 2026

End of SDLC Document